Database-Focused System Design Interview Framework

Table of Contents

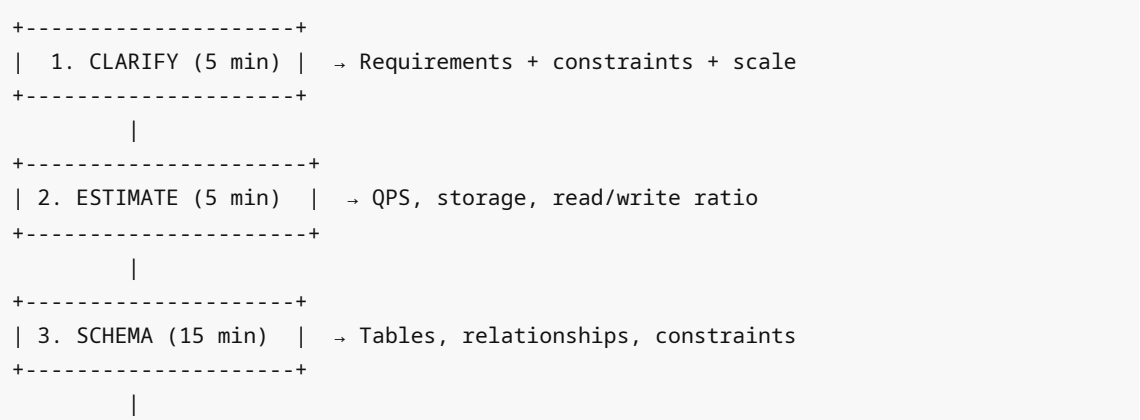
Overview

Database-focused system design interviews test your ability to translate business requirements into a production-ready data architecture. Unlike general system design (which focuses on services, APIs, and infrastructure), DB-focused interviews go deep on:

- Table structure, constraints, and normalization decisions
- Index selection and query performance
- Consistency requirements and transaction isolation
- Scaling patterns specific to PostgreSQL
- Tradeoffs between competing design goals

This framework gives you a repeatable process for answering any database system design question in 45–60 minutes.

The Six-Step Framework



```
+---------------------+
| 4. INDEXES (5 min)  |  → Which indexes, which type, why
+---------------------+
         |
+---------------------+
| 5. SCALING (10 min) |  → Replicas, partitions, sharding, cache
+---------------------+
         |
+---------------------+
| 6. TRADEOFFS (5 min)|  → Every decision has a cost
+---------------------+
```

Time allocation is approximate; adjust based on the interviewer's emphasis.

Step 1: Clarify Requirements

Questions to Always Ask

Scale questions:

- How many users? (active vs. registered)
- What is the expected QPS? Peak vs. average?
- How much data needs to be stored? For how long?

Functional scope:

- Which features are must-have for MVP?
- Which features can be simplified or deferred?
- Are there existing systems we're integrating with?

Consistency and correctness:

- Can reads be slightly stale? How stale is acceptable?
- Are there operations that must be atomic? (money transfers, inventory)
- What happens on partial failure? (retry, compensate, or fail hard?)

Compliance and retention:

- Any regulatory requirements? (GDPR, HIPAA, PCI-DSS)
- How long must data be retained?
- Are there audit requirements?

Clarification in Practice

Bad approach:

> *"I'll design the schema for a URL shortener."*

Good approach:

> *"Before I start, let me clarify a few things. Are we building this from scratch or adding to an existing system? How many short URLs do we expect to create per day — around 1M? And for redirects, are we optimizing for latency (caching at CDN level) or accuracy (we need to log every click)? Also, do we need custom domains and user accounts, or is this an anonymous service?"*

Stating Assumptions

After clarifying, explicitly state your assumptions:

> *"I'll assume: 1M URL creations per day, 100M redirects per day, users can have accounts to manage their URLs, and we need click analytics. I'll design for these numbers and call out where the design changes at 10x scale."*

Step 2: Capacity Estimation

The Three Numbers

Always compute these three before designing:

Number	How to compute	Purpose
QPS (reads + writes)	Daily requests / 86,400	Determines concurrency requirements
Storage per year	(Row size × rows/day) × 365	Determines partitioning strategy
Read:Write ratio	Reads / Writes	Determines caching and replica strategy

Estimation Shortcuts

QPS from DAU:

- Assume 10 actions per DAU per day average
- Peak is ~3x average for most consumer apps

Storage from events:

```
Storage = (rows/day × avg_row_bytes × 365) / 1B bytes per GB
Example: 1M orders/day × 300B/row × 365 = ~100GB/year
```

Useful conversions:

```
1M rows/day = ~12 rows/second
1B rows/day = ~11,600 rows/second = ~12K QPS
1TB = 1 billion × 1KB records
100GB/year ≈ 270MB/day ≈ 3KB/second
```

Sample Estimation Walkthrough (URL Shortener)

```
Given: 1M URL creations/day, 100M redirects/day

Write QPS: 1M / 86,400 = ~12 writes/sec (easy for PostgreSQL)
Read QPS: 100M / 86,400 = ~1,160 reads/sec (needs caching)
Read:Write ratio = 100:1 → highly read-optimized design

Storage (short_urls):
  1M rows/day × 600B/row × 365 days = ~219GB/year
  → Single PostgreSQL instance handles this easily

Storage (click_events):
```

```
  100M rows/day × 200B/row = 20GB/day = 7.3TB/year
  → Partitioned table, archive after 90 days
  → Need 90 × 20GB = 1.8TB hot storage

Cache requirement:
  100M active URLs × 600B = 60GB → fits in Redis
```

Step 3: Schema Design

Design Process

Step 3a: Identify entities List the nouns in your requirements. Each noun is probably a table.

Step 3b: Define relationships

- 1:N (user has many orders)
- M:N (products have many categories; categories have many products)
- 1:1 (order has one payment)

Step 3c: Write CREATE TABLE statements Start with the 3–5 most important tables. Add constraints and defaults.

Step 3d: Draw the ER diagram Even a rough ASCII diagram communicates the structure.

Schema Design Checklist

For each table, verify:

- ☐ Primary key chosen (UUID vs. BIGSERIAL — know the tradeoffs)
- ☐ NOT NULL constraints on required fields
- ☐ UNIQUE constraints on natural keys
- ☐ Foreign key constraints with appropriate ON DELETE behavior
- ☐ CHECK constraints for domain validation
- ☐ Timestamps: `created_at` always NOT NULL DEFAULT now(); `updated_at` maintained by trigger
- ☐ Enums vs. VARCHAR for status fields (prefer ENUM for stability)
- ☐ JSONB vs. normalized columns for variable data

Key Decisions to Explain

UUID vs. BIGSERIAL:

- UUID: globally unique (no coordination), harder to guess (security), larger (16B vs 8B), random UUIDs fragment B-tree indexes (use UUIDv7 for time-ordered)
- BIGSERIAL: sequential (good for B-tree), compact, requires central sequence, enumerable

Normalization tradeoffs:

- Normalized: consistent, no redundancy, easier writes
- Denormalized: faster reads, simpler queries, risk of inconsistency
- Rule: normalize first; denormalize only for measured bottlenecks

Partitioning decision: When to partition: table > 100GB AND queries have a natural time filter AND you want to purge old data

When to use JSONB: For attributes that vary per row (product attributes, event properties, user settings), JSONB avoids an EAV antipattern and is indexable via GIN.

Common Schema Patterns

```
-- Status field: always use ENUM, not VARCHAR
CREATE TYPE order_status AS ENUM ('pending', 'confirmed', 'shipped', 'delivered',
'cancelled');

-- Soft delete
ALTER TABLE items ADD COLUMN deleted_at TIMESTAMPTZ;
-- Query: WHERE deleted_at IS NULL

-- Audit columns
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
-- Add trigger: CREATE TRIGGER ... tsvector_update_trigger ...

-- Idempotency key for payment/order APIs
idempotency_key VARCHAR(100) UNIQUE

-- M:N relationship
CREATE TABLE post_tags (
    post_id UUID REFERENCES posts(post_id) ON DELETE CASCADE,
    tag_id  INT  REFERENCES tags(tag_id)   ON DELETE CASCADE,
    PRIMARY KEY (post_id, tag_id)
);

-- Insert-only upsert
INSERT INTO counts (id, count) VALUES ($1, 1)
ON CONFLICT (id) DO UPDATE SET count = counts.count + 1;
```

Step 4: Indexing Strategy

The Decision Tree

```
What type of query?
│
├── Exact match on one column → B-Tree single column
├── Exact match + range on another column → B-Tree composite (equality first, range
last)
├── Full-text search (LIKE '%word%') → GIN with pg_trgm OR tsvector
├── JSONB containment (@>) → GIN on JSONB column
├── Array containment (@>) → GIN on array column
├── Spatial query (ST_DWithin) → GiST on geography column
├── Exclusion constraint (no overlapping ranges) → GiST
├── Append-only time-series → BRIN on timestamp column
└── Sparse filter on large table → Partial index with WHERE clause
```

Index Design Rules

1. **Composite index column order**: Put equality columns first, range/sort columns last

```
-- Query: WHERE user_id = $1 AND status = 'active' ORDER BY created_at DESC
-- Index: (user_id, status, created_at DESC)  ← correct order
```

2. **Partial indexes for sparse conditions**:

```
-- Instead of indexing all 10M rows:
CREATE INDEX idx_pending ON orders (user_id, created_at) WHERE status =
'pending';
-- Only indexes the 1% of rows that matter
```

3. **Covering indexes to enable index-only scans**:

```
CREATE INDEX idx_users_email ON users (email) INCLUDE (user_id, display_name);
-- SELECT user_id, display_name FROM users WHERE email = $1 → no heap access
```

4. **Don't over-index writes**: Every index you add slows down INSERT/UPDATE/DELETE. On high-write tables, 3–5 indexes maximum.

Index Types Quick Reference

Type	Use Case	Size	Write Overhead
B-Tree	Equality, range, ORDER BY, LIKE prefix	Medium	Low
GIN	FTS, array, JSONB containment	Large	High
GiST	Geometry, range types, exclusion	Medium	Medium
BRIN	Monotonic append-only data	Very small	Very low
Hash	Equality only	Small	Low

Step 5: Scaling Strategy

The Scaling Ladder

Present scaling as a progression. You start simple and add complexity only when the previous level is insufficient.

```
Level 1: Single primary PostgreSQL
         → Handles up to ~10K reads/sec, ~5K writes/sec
         → Simple; start here

Level 2: + Connection pooling (PgBouncer)
         → Handles connection explosion from application scaling
         → Cost: near zero; benefit: prevents connection overload
```

```
Level 3: + Read replica(s)
         → Scales reads horizontally
         → Cost: replication lag; must handle read-your-own-writes
         → When: read:write ratio > 5:1

Level 4: + Caching layer (Redis)
         → Absorbs repeated reads for hot data
         → When: specific hot rows or queries are overwhelming the replica

Level 5: + Table partitioning
         → Manages table size; enables fast data purge
         → When: table > 100GB with time-based queries

Level 6: + Functional decomposition (separate DB per service)
         → Independent scaling per domain
         → When: one domain is overwhelming the shared instance

Level 7: + Horizontal sharding (Citus, manual)
         → Scales write capacity
         → When: write QPS exceeds what a single primary can handle

Level 8: + Polyglot persistence
         → Purpose-built stores for purpose-built workloads
         → When: PostgreSQL is the wrong tool for a specific workload
```

When to Use Each Technique

Technique	Apply when	Cost
Index optimization	Any query > 100ms	Low
Connection pooling	> 100 app instances	Low
Read replicas	Read QPS > 10K	Medium (replication lag)
Query caching (Redis)	Same data read 100+/sec	Medium (invalidation complexity)
Partitioning	Table > 100GB with time filter	Low operationally
Materialized views	Complex aggregation used frequently	Low (stale data window)
Sharding	Write QPS > 50K	High (cross-shard queries)
Polyglot (Elasticsearch)	FTS on > 50M rows	High (sync complexity)

Scaling Narrative Template

> *"At the scale we estimated (12K writes/sec, 1M reads/sec), a single PostgreSQL instance cannot serve all reads. Here's the architecture I'd use:*
>
> ***Write path**: All writes go to the primary PostgreSQL. At 12K writes/sec, the primary handles this with PgBouncer transaction-mode pooling. Partition the events table by day to manage data lifecycle.*

> ***Read path**: 95% of reads hit Redis (URL lookups are the same URLs repeated many times). The remaining 5% cache misses hit a read replica. We have one replica for user-facing reads and one for analytics/reporting.*
>
> ***At 10x scale**: The Redis cache would still absorb the increased read load. We'd add Redis cluster nodes. For writes (120K/sec), we'd introduce sharding by hash(url_id) % 4, giving 4 shards of 30K writes/sec each — within PostgreSQL's comfortable range."*

Step 6: Tradeoffs Discussion

Tradeoff Framework

Every design decision has:

1. **What you gain** — the benefit
2. **What you give up** — the cost
3. **When the tradeoff is worth it** — the threshold

Common PostgreSQL Tradeoffs

Decision	Gain	Cost	Worth it when...
UUID primary key	Distributed inserts, no enumeration	Larger index, random I/O, slower sequential	Distributed system or security concern
Read replica	More read throughput	Replication lag, split-brain risk	Read:write > 5:1
Denormalized count column	O(1) count reads	Risk of inconsistency, trigger overhead	COUNT(*) is on hot path
Table partitioning	Fast purge, partition pruning	Management overhead, some query complexity	Table > 100GB with time filters
JSONB column	Schema flexibility, no ALTER TABLE	No FK enforcement, harder to query specific fields, larger indexes	Truly variable attributes
SERIALIZABLE isolation	No write skew	Higher abort rate, slightly lower throughput	Money transfers, inventory
Materialized view	Sub-millisecond for complex queries	Stale data, REFRESH CONCURRENTLY needed	Query takes > 1s, freshness lag OK

Must-Know Tradeoffs for Interviews

1. Consistency vs. Availability (CAP Theorem) With PostgreSQL and synchronous replication: during a network partition, the primary continues to serve writes (CP system). Without synchronous replication, we get higher availability but risk data loss (AP).

2. Normalization vs. Performance Normalized: `SELECT p.title, c.name FROM products p JOIN categories c ON ...` — consistent, but JOINs are slower. Denormalized: `products.category_name VARCHAR` — fast reads, but inconsistent when category name changes.

3. Pessimistic vs. Optimistic Locking Pessimistic (`SELECT FOR UPDATE`): safe, simple, but rows are locked for the transaction duration. Optimistic (version column): higher throughput, no locks, but requires retry on

conflict.

4. Fan-out-on-write vs. Fan-out-on-read (Social Feed) Write: fast reads, expensive writes (O(followers) writes per post). Read: cheap writes, expensive reads (O(follows) JOINs per feed load). Hybrid: write for regular users, read for celebrities.

Common Anti-Patterns to Avoid

Schema Anti-Patterns

1. **EAV (Entity-Attribute-Value) tables**: `(entity_id, attribute_name, attribute_value)` rows. Unmaintainable, slow, no type safety. Use JSONB instead.
2. **Storing comma-separated lists in a column**: `tags VARCHAR = 'sql,postgres,db'`. Use an array column or a junction table.
3. **Using VARCHAR(MAX) everywhere**: Choose appropriate lengths. Unlimited length strings increase storage and make query estimates less accurate.
4. **Storing calculated values without enforcing consistency**: `total = price * quantity` stored in a column. Use `GENERATED ALWAYS AS` or a trigger, never rely on application code alone.
5. **No timestamps**: Every table should have `created_at`. Tables with UPDATE semantics need `updated_at`.

Query Anti-Patterns

6. **SELECT * in application code**: Pulls unnecessary columns, prevents index-only scans, breaks when schema changes.
7. **OFFSET-based pagination on large tables**: `OFFSET 100000 LIMIT 50` requires scanning 100,050 rows to discard 100,000. Use cursor-based pagination.
8. **N+1 queries**: Loading 100 posts then querying for each post's author. Use JOIN or batch query.
9. **Unbounded queries**: `SELECT * FROM orders WHERE user_id = $1` — returns all orders ever. Always include LIMIT.
10. **OR conditions killing indexes**: `WHERE status = 'active' OR user_id = $1` can prevent index use. Use UNION ALL instead.

Scaling Anti-Patterns

11. **Starting with sharding**: Introduces cross-shard complexity before you need it. Start with vertical scaling and partitioning.
12. **Read replicas for write-heavy workloads**: Replicas only help reads. If your bottleneck is write QPS, replicas don't help.
13. **Ignoring autovacuum**: High UPDATE/DELETE rate + disabled autovacuum = table bloat, slower queries, eventually table lockups.
14. **Giant transactions**: Long-running transactions hold locks, block autovacuum, and increase rollback time on failure. Break into smaller batches.

15. **Not testing with production data volumes**: An index on 10K rows works fine. The same query on 100M rows may need a completely different approach.

Quick Numbers Reference

PostgreSQL Throughput (single node, modern hardware)

Operation	Throughput
Simple index reads (PK lookup)	50K–100K reads/sec
Simple writes (INSERT)	10K–50K rows/sec
COPY bulk insert	100K–1M rows/sec
Complex analytical queries	1–10 queries/sec
LISTEN/NOTIFY channels	~10K active
Max practical connections	100–500 (use PgBouncer above)
Max table size	32TB
Max row size	1.6TB
Max index key size	8KB

Latency Reference

Operation	P50	P99
PostgreSQL index read (hot cache)	0.5ms	2ms
PostgreSQL index read (cold)	5ms	50ms
PostgreSQL INSERT (single)	1ms	5ms
Redis GET (in-memory)	0.1ms	0.5ms
pg_notify delivery (same machine)	<1ms	2ms
Elasticsearch query	5ms	50ms
Network round-trip (same DC)	0.5ms	2ms
Network round-trip (cross-region)	50–200ms	400ms

Storage Sizes

Data Type	Size
INT / SERIAL	4B
BIGINT / BIGSERIAL	8B

UUID	16B
TIMESTAMPTZ	8B
CHAR(2)	2B
VARCHAR(100)	1–100B + overhead
JSONB (empty)	4B
GEOMETRY point	48B
GEOGRAPHY point	48B + 20B overhead

System-Specific Checklists

E-Commerce Checklist

- [ ] Inventory reservation (not just decrement) — handles concurrent checkout
- [ ] Idempotency key on payments — prevents double charges
- [ ] Order state machine — defined transitions, not free-form updates
- [ ] Product variant table — handles size/color combinations
- [ ] Soft delete products — don't lose historical order references
- [ ] Orders partitioned by date — enables archival

Banking Checklist

- [ ] Double-entry ledger — debit = credit invariant
- [ ] SERIALIZABLE isolation for transfers — prevents write skew (overdraft)
- [ ] Idempotency key on transactions — prevents double processing
- [ ] Immutable transaction records — corrections via reversals only
- [ ] Audit trigger on sensitive tables — every change logged
- [ ] `synchronous_commit = on` — no data loss on crash
- [ ] Balance snapshots — enables point-in-time balance queries

Social Network Checklist

- [ ] Follows as directed graph — follower/followee distinction
- [ ] Feed strategy defined — push vs. pull vs. hybrid
- [ ] Celebrity threshold identified — users above N followers get pull feed
- [ ] Post partitioned by date — billions of rows require partitioning
- [ ] Denormalized like/comment counts — avoid COUNT(*) at page load
- [ ] Soft delete posts — notifications/likes reference deleted posts

Chat Application Checklist

- [ ] DM dedup — canonical ordering (user_a < user_b) prevents duplicates
- [ ] Message sequence number per conversation — ordering + sync
- [ ] LISTEN/NOTIFY trigger on messages — real-time delivery
- [ ] Presence as UNLOGGED table — ephemeral, fast writes

- [ ] Cursor-based pagination — not OFFSET
- [ ] client_msg_id deduplication — idempotent sends
- [ ] Partitioned messages — daily partitions, archive after 90 days

Analytics Platform Checklist

- [ ] Events partitioned by day — for fast purge and pruning
- [ ] BRIN index on occurred_at — small size for monotonic data
- [ ] Pre-aggregated hourly/daily tables — dashboard reads from aggregates, not raw events
- [ ] Materialized views for complex summaries — pre-compute funnels
- [ ] Kafka buffer before PostgreSQL — absorbs 2M events/sec ingest
- [ ] Separate OLAP store for historical queries — columnar for cold data

---

Practice Problems

Problem 1: Design a Waitlist System

A concert ticketing platform has events that sell out. Users can join a waitlist and are notified when seats become available. Design the database.

Key entities: events, seats, waitlist_entries, notifications **Key challenge**: When a seat opens, notify users in order (FIFO) without double-booking **Solution hint**: `SELECT ... FOR UPDATE SKIP LOCKED` on the waitlist queue

---

Problem 2: Design a Poll/Survey System

Users can create polls with multiple options. Respondents choose one option (for single-choice) or multiple (for multi-choice). Real-time vote counts displayed.

Key entities: polls, options, responses **Key challenge**: Real-time vote counts at high volume (viral poll) **Solution hint**: Redis INCR for real-time counts; PostgreSQL for storage and exact counts

---

Problem 3: Design an Inventory System for a Warehouse

Multiple warehouses, each with thousands of SKUs. Orders reserve inventory, then fulfill it. Restocking happens regularly.

Key entities: warehouses, skus, inventory, reservations, inventory_transactions **Key challenge**: Concurrent reservations for the same SKU **Solution hint**: Row-level locking with `FOR UPDATE` ; generated column for available = on_hand - reserved

---

Problem 4: Design a Feature Flag System

Feature flags can be toggled per user, per organization, or by percentage rollout. Each flag can have a boolean, string, or JSON value.

Key entities: feature_flags, flag_overrides, rollout_rules **Key challenge**: Sub-millisecond evaluation; runtime changes without deployment **Solution hint**: DB is source of truth; Redis cache per org with 60-second TTL

Interview Tips

Presentation Tips

1. **Think out loud**: Share your reasoning. "I'm considering two approaches — a joined table or JSONB. Let me think through the access patterns... Given that we filter by these properties frequently, I'll use normalized columns."
2. **Start simple, then complicate**: Propose the simple design first. Then say "at 10x scale, this breaks because... here's how I'd evolve it."
3. **Draw as you speak**: Even ASCII diagrams on paper help. Spatial representation of tables helps interviewers follow your design.
4. **Name your constraints**: "I'm adding a CHECK constraint here because the business rule requires..." shows you understand why, not just what.
5. **Call out what you're skipping**: "I'm not designing the authentication tables for now, but in production we'd need users, sessions, and tokens."

What Interviewers Are Looking For

Signal	Demonstrated By
Business understanding	Asking clarifying questions about actual requirements
Data modeling instinct	Correct normalization level for the use case
Index knowledge	Choosing index type based on query pattern, not convention
Scale intuition	Knowing PostgreSQL's limits and when each technique is needed
Tradeoff awareness	Naming what you give up with every choice
Practical experience	Mentioning VACUUM, connection pooling, partition management

Common Interview Mistakes

1. **Jumping to NoSQL immediately**: For most problems, start with PostgreSQL and justify a move to NoSQL if required. Interviewers want to see you know relational modeling first.
2. **Forgetting constraints**: Tables without foreign keys, CHECK constraints, or UNIQUE constraints signal lack of production experience.
3. **Proposing Elasticsearch for everything**: Use PostgreSQL FTS for up to ~50M rows. Elasticsearch adds operational complexity; only introduce it when justified.
4. **Not discussing VACUUM/bloat**: For high-update tables, not mentioning autovacuum tuning signals lack of PostgreSQL production experience.
5. **Designing for 10B users from day 1**: Over-engineering signals lack of pragmatism. Show you can scale the design progressively.

Cross-References

- Amazon case study: `18_Architecture_Case_Studies/01_amazon_marketplace_db.md`
- Netflix case study: `18_Architecture_Case_Studies/02_netflix_streaming_platform.md`

- Uber case study: `18_Architecture_Case_Studies/03_uber_ridesharing.md`
- Banking case study: `18_Architecture_Case_Studies/04_banking_platform.md`
- SaaS case study: `18_Architecture_Case_Studies/05_saas_product.md`
- Messaging case study: `18_Architecture_Case_Studies/06_messaging_system.md`
- Architecture interview Q&A: `18_Architecture_Case_Studies/07_architecture_interview_guide.md`
- URL Shortener design: `01_url_shortener.md`
- E-Commerce design: `02_ecommerce_system_design.md`
- Banking design: `03_banking_system_design.md`
- Ride-sharing design: `04_ride_sharing_design.md`
- Social network design: `05_social_network_design.md`
- Chat application design: `06_chat_application_design.md`
- Analytics platform design: `07_analytics_platform_design.md`